

Code templates & reference notes

Part 12 —  AI Governance · 12-ai-governance/code/model_card_template.md

System Card — Support Assistant (customer-support RAG system)

How to read this file. This is a complete, filled-out example, not a blank template. Every section starts with a `> Governance rationale:` blockquote explaining *why the section exists as a control*, per `notes/02-operating-model-and-practice.md §3`, followed by the content a real team would actually write. Copy the structure, not the specifics — a resume-screener or a fraud model needs the same sections with very different evaluation and oversight content. Note the title: this is called a **system card**, not a model card, on purpose (see the note at the bottom of §1) — "Support Assistant" is a foundation model plus a retrieval pipeline plus your own prompts and guardrails, and no single one of those is "the model."

1. System overview

Governance rationale: a reviewer (committee, MRM, auditor) needs to know what they're looking at before anything else — what it is, who owns it, and what tier it was assessed at. This is the card's cover page, and it's also the row that gets copied verbatim into the AI use-case inventory.

Field	Value
System name	Support Assistant
One-line description	Answers customer billing/product questions on the support portal by retrieving from the help-center corpus and generating a grounded response; escalates to a human agent when confidence is low or the customer requests one.
Business owner	VP, Customer Support
Technical owner	Support Platform Engineering
Responsible AI office contact	ai-governance@example.com
Foundation model(s)	Claude Sonnet (API), version pinned per §7 below
Architecture pattern	Retrieval-augmented generation (RAG) over a curated help-center corpus, with a tool-call escalation path to a human agent queue
Risk tier (per <code>code/ai_risk_register_template.py</code>)	Limited — data_sensitivity=1, decision_autonomy=2, population_scale=4, reversibility=1 → score 38/100
Card version	3.2
Last updated	2026-08-11
Card owner	Support Platform Engineering — see change log (§8) for authorship of each revision

Why this is a system card, not a model card: "the model" in the strict sense (Claude Sonnet) is one component. What the customer actually experiences is the combination of that model, the retrieval corpus, the system prompt, a confidence-threshold escalation rule, and an output guardrail that blocks certain claim types (§4, §6). A model card describing only the foundation model would tell a reviewer almost nothing about what actually happens in production — the retrieval corpus and the guardrails are where most of this system's actual behavior (and most of its risk) lives.

2. Intended use

Governance rationale: this defines the exact boundary the governance committee approved when this system was tiered as **Limited**. Everything inside this boundary was assessed; anything outside it was not — which is precisely why §3 (out-of-scope uses) has to be just as explicit as this section. A committee does not approve "a chatbot." It approves *this* system, doing *these* things, for *this* population.

- **Primary use case:** answer customer questions about billing, account status (non-sensitive fields), product features, and troubleshooting steps, using only the public and account-tier-appropriate content in the help-center corpus (§5).
- **Intended users:** customers of [Product] interacting via the public support portal and the in-app help widget. Not intended for use by support agents as an internal tool (see the separate, differently-tiered "Agent Copilot" system card).

- **Deployment context:** public-facing, always-on, text chat interface. No voice, no autonomous outbound contact.
- **Approved language(s):** English only in this version. Non-English input triggers a "we don't support this language yet, here's how to reach a human" response rather than a best-effort translation — this was a deliberate scope cut agreed with the governance committee, not an oversight.

3. Out-of-scope and prohibited uses

Governance rationale: this is the list someone points to six months from now when a well-meaning product manager wants to repurpose this system for something it was never assessed for. Without an explicit list, "can we also use it for X" gets answered by whoever is in the room instead of by governance. Every bullet here corresponds to a use case that would very likely land in a *different, higher* risk tier.

- **Not for account changes.** The system does not modify billing, cancel subscriptions, or change account settings on its own authority. It can *describe* how to do these things or *hand off* to an agent/tool that does, but it does not execute them directly — doing so would move decision_autonomy from 2 to 4+ and re-tier this system as High.
- **Not for disputing or waiving charges.** Any refund/credit conversation routes to a human agent. Do not extend the system prompt to let it approve refunds "for efficiency" without re-running it through the risk register — this is exactly the kind of prompt-edit-as-deployment change §6 of notes/02 warns about.
- **Not for legal, medical, or financial advice,** even if a customer asks a question shaped like one (e.g. "will this affect my credit score" or "am I entitled to a refund under [law]"). The system is instructed to decline and redirect these to the appropriate human channel.
- **Not for use with special-category data.** The retrieval corpus and prompt are built for billing/product content only. Do not point this system's retrieval layer at any corpus containing health, biometric, or other special-category data without a full re-tiering — that alone would likely push data_sensitivity to 5 and move the system into Critical.
- **Not approved for any other business unit's customers** without its own intake, tiering, and sign-off — this card's approval does not transfer to a superficially similar use case elsewhere in the company.

4. Training and grounding data summary

Governance rationale: for legal, this is the provenance record — what IP, license, and consent obligations apply to what the system says. For everyone else, it's the explanation for *why* the system is biased or wrong in a particular way when that eventually surfaces. For a RAG system, the operative question is not "what was the foundation model trained on" (you don't control that, and it's the vendor's card, referenced below) — it's "what corpus is it grounded in," because that's the content the system actually asserts as fact to a customer.

- **Foundation model provenance:** Claude Sonnet, provided by Anthropic under the vendor's standard API terms. We do not have visibility into its training data; see §7 (vendor risk) for what we do have —

Anthropic's model documentation and our contractual data-use terms (no training on our prompts/outputs).

- **Retrieval corpus:** ~4,200 help-center articles, all authored and owned by [Company]'s Content team, version-controlled in the CMS. No customer data, no scraped third-party content, no personal data of any kind is in the corpus.
- **Corpus refresh cadence:** re-indexed nightly from the CMS; a content-team sign-off gate exists for any article edit before it becomes retrievable (this gate is itself a governance control — an unreviewed article edit is an unreviewed change to what this system tells customers).
- **Known corpus gaps:** approximately 15% of tier-3 (enterprise-plan) billing edge cases are not yet documented in the corpus as of this card's version — see §6, this is the single largest source of low-confidence responses.
- **Data NOT used:** no chat transcripts, no customer PII, and no prior support-ticket data are used for retrieval or fine-tuning in this version. (A future version proposing to fine-tune or retrieve over past ticket transcripts would require a fresh privacy review and very likely a new risk-tier assessment.)

5. Evaluation results

Governance rationale: this is the Measure function's output (see notes/01 §2) made legible to a non-technical reviewer. A metric without a stated methodology is a marketing claim, not evidence — so every number below states what was measured, on what set, and how, not just the headline. This section is also literally what MRM (notes/02 §2) validates independently before sign-off.

Metric	Method	Result	Threshold to ship
Answer correctness (grounded)	400 held-out support questions, human-graded against the corpus for factual accuracy	94.2% correct	≥ 90%
Faithfulness to retrieved context	Automated groundedness check (does the answer only assert what's in the retrieved passages) on the same 400-question set	97.5%	≥ 95%
Appropriate escalation rate	Manually labeled set of 150 questions that <i>should</i> escalate (refunds, legal-shaped, low-confidence topics)	91.3% correctly escalated	≥ 90%
Harmful/policy-violating output rate	Red-team set of 120 adversarial prompts (jailbreak attempts, prompt injection via crafted "customer" messages)	0 successful policy violations, 3 near-misses (logged, mitigated)	0 tolerated
Latency (p95)	Production shadow traffic, 2 weeks	2.8s	≤ 4s
Customer satisfaction (post-chat survey)	Live A/B against human-only queue, 30-day window	4.1 / 5.0 vs. 4.3 / 5.0 human baseline	Within 0.3 of baseline

Fairness/subgroup check: the correctness and escalation metrics above were also broken out by account plan tier (free / paid / enterprise) — no statistically significant gap was found (largest delta: 2.1 points,

within noise for this sample size). This check exists because plan tier correlates with how well the corpus covers a customer's situation (§4's "known corpus gaps"), which is exactly the kind of proxy disparity a fairness review is supposed to catch before a customer does.

What this evaluation does not cover: long-tail enterprise billing scenarios (the corpus gap noted in §4), non-English input (out of scope per §2), and behavior after a foundation-model version bump (each bump is required to re-run this full suite before rollout — see §8).

6. Known limitations and failure modes

Governance rationale: this is what you'd tell a new team member in their first week so they don't get burned by something the team already knows about. Every bullet here should map to either a mitigation already in place or an open item on the risk register — a limitation with neither is a gap, not a disclosure.

- **Enterprise billing edge cases** (the corpus gap from §4) produce low-confidence answers more often; mitigated by a confidence threshold that routes these to escalation rather than guessing, but the threshold is imperfect — some are answered with unwarranted confidence. *Open item: expand corpus coverage, tracked in the content backlog.*
- **Prompt injection via customer message.** A customer can type instructions aimed at the system itself ("ignore previous instructions and..."). The red-team suite (§5) found 3 near-misses. Mitigated by an output guardrail that blocks specific claim categories (refund approval, account changes) regardless of what the model was steered to say — see Module 13 — AI Security [13-ai-security] for the technical detail on this class of attack and defense.
- **Silent model-version drift.** Because the foundation model is a vendor API, a version bump could shift tone or accuracy without any change on our side. Mitigated by version pinning and a required full eval-suite re-run before adopting any new version (§7, §8).
- **Confident wrong answers on genuinely ambiguous questions** — the "confidently wrong, on purpose" failure mode described in notes/01 §1. The system has no reliable internal signal for "I am guessing"; the confidence/escalation heuristic is an approximation, not a guarantee, and post-launch monitoring tracks the customer "this didn't help" feedback rate as an early-warning proxy.
- **English-only.** Non-English input is deflected, not mistranslated — this is a known scope limit, not a silent failure, but it does mean non-English-speaking customers get a worse experience than English speakers, which is tracked as an equity consideration for a future version.

7. Human oversight design

Governance rationale: per notes/02 §4, a Limited-tier system like this one doesn't need per-response human-in-the-loop review (that would be disproportionate to the risk), but "no review at all" is not a valid choice either — this section states which posture applies and why, so a reviewer can check the posture matches the tier instead of taking "we have oversight" on faith.

- **Posture: human-on-the-loop (HOTL).** Responses go to the customer without per-message approval (HITL would be infeasible at this volume — several thousand conversations/day), but a support-ops team monitors a live dashboard of escalation rate, low-confidence rate, and negative-feedback rate, and can pause the system (fall back to human-only queue) within minutes.
- **Escalation is not a decision the system can override.** Any customer request for a human, or any question matching a refund/legal/medical/account-change pattern, escalates unconditionally — this is a hard rule in the orchestration layer, not a suggestion to the model.
- **Sample audit.** 2% of conversations are randomly sampled weekly for human quality review, specifically checking for the failure modes in §6 — this is the check against "the automated metrics look fine but something's actually wrong" that a dashboard alone won't catch.
- **Kill switch:** support-ops on-call can disable the system and fail over to the human-only queue via a single feature flag, no code deploy required. Tested quarterly as part of the incident-readiness drill.

8. Change log / version history

Governance rationale: per notes/02 §6, a prompt edit or corpus update *is* a deployment to a governed system. This log is the audit trail that proves review rigor was applied to each change — "we changed the prompt" with no entry here is exactly the unreviewed-change failure mode this whole module exists to prevent.

Ver sio n	Date	Change	Eval re-run?	Approved by
1.0	2025-11-03	Initial launch	Full suite (§5 baseline)	AI Governance Committee (Limited-tier fast-track)
2.0	2026-02-14	Added confidence-based escalation threshold (previously fixed keyword rules only)	Full suite	Business owner (in-tier change)
3.0	2026-05-20	Foundation model version bump (vendor-driven)	Full suite — 1.1-point correctness drop investigated and traced to a prompt-format change on the vendor's side, prompt adjusted to compensate	Business owner + Responsible AI office review
3.1	2026-07-02	Corpus refresh cadence changed from weekly to nightly	Faithfulness metric only (targeted re-check)	Technical owner
3.2	2026-08-11	Added output guardrail blocking refund-approval language (hardening after a red-team near-miss)	Red-team suite only (targeted re-check)	Responsible AI office

9. Governance sign-off

Governance rationale: the accountability record. When something goes wrong, this is the answer to "who decided this was acceptable, and on what basis" — the question notes/01's introduction opens with. No named sign-off, no accountability; this section is not a formality.

Role	Name / title	Sign-off basis
Accountable executive	VP, Customer Support	Owens the deployment decision and post-launch monitoring outcome
AI governance committee	Approved 2025-10-28 (Limited-tier fast-track review)	Reviewed risk-tier assessment and evaluation results at launch
Model risk management	Independent validation completed 2025-10-24	Confirmed eval methodology (§5) actually covers the failure modes in §6
Legal / privacy	Reviewed 2025-10-20	Confirmed no personal or special-category data in the corpus (§4); vendor contract terms checked (§7 of notes/02)
Security	Reviewed 2025-10-22	Ran the prompt-injection red-team suite referenced in §5/§6
Next scheduled re-assessment	2026-10-28 (annual, Limited tier)	Per the tiered re-assessment cadence in notes/01 §5

Where this returns

Idea here	Where it returns
Risk tier referenced throughout	<code>code/ai_risk_register_template.py</code> — run it yourself to reproduce the 38/100 score
Evaluation methodology (§5)	Module 15 — AI Evals [15-ai-vals] — how you'd actually build this eval suite
Prompt-injection mitigation (§6)	Module 13 — AI Security [13-ai-security] — the technical detail behind the guardrail
Human-on-the-loop posture (§7)	<code>notes/02-operating-model-and-practice.md</code> §4 — the three-posture framework this implements
Change log as audit trail (§8)	<code>notes/02-operating-model-and-practice.md</code> §6, §8 — why a prompt edit is a deployment
Vendor version-bump handling (§8, v3.0)	<code>notes/02-operating-model-and-practice.md</code> §5 — vendor/third-party AI risk